

Colab Agent

Autonomous agentic interface for fine-tuning HuggingFace models in Google Colab. Uses an LLM-powered loop to plan, generate, execute, and iterate on fine-tuning workflows inside Colab notebooks — with runtime auto-switching, error recovery, cost tracking, persistent storage, a plugin system, and a self-improvement loop.

502 tests pass across all phases (4 pre-existing infra-dependent failures).

Quickstart

```
# Install
pip install -e .

# Configure (copy and fill in)
cp .env.example .env

# Initialize DB
colab-agent init

# Run a goal
colab-agent run "Fine-tune Phi-2 for code generation"

# Interactive mode
colab-agent interactive

# Serve HTTP API
colab-agent serve
```

Configuration

All settings via `.env` file or `COLAB_AGENT_*` environment variables:

Variable	Description	Default
COLAB_AGENT_LLM_PROVIDER	openai, anthropic, gemini, local	openai
COLAB_AGENT_LLM_MODEL	Model name	gpt-4
COLAB_AGENT_OPENAI_API_KEY	API key	—
COLAB_AGENT_OPENAI_BASE_URL	Custom endpoint (e.g. NVIDIA API)	—
COLAB_AGENT_HF_TOKEN	HuggingFace token	—
COLAB_AGENT_GITHUB_TOKEN	GitHub personal access token	—
COLAB_AGENT_GITHUB_REPO	Target repo for error push	—
COLAB_AGENT_DEFAULT_FINETUNE_METHOD	lora, qlora, full	qlora
COLAB_AGENT_DEFAULT_BASE_MODEL	Base model ID	microsoft/phi-2

Variable	Description	Default
COLAB_AGENT_ENV	Config profile: dev, staging, prod	dev
COLAB_AGENT_API_KEY	Required when ENV=prod	—
COLAB_AGENT_DATABASE_URL	PostgreSQL URL (auto-detects SQLite vs PG)	sqlite:///...
COLAB_AGENT_BUDGET_MAX_UNITS	Max cost units before abort	100
COLAB_AGENT_BUDGET_WARN_THRESHOLD	Alert at % of budget	80

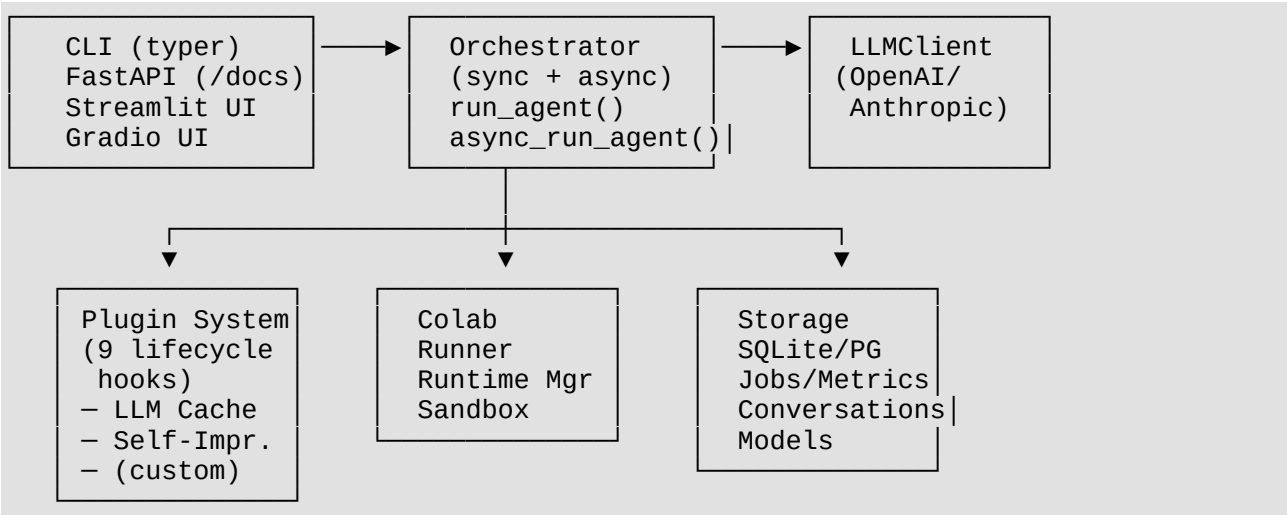
NVIDIA API Example

```
COLAB_AGENT_LLM_PROVIDER=openai
COLAB_AGENT_LLM_MODEL=meta/llama-3.1-8b-instruct
COLAB_AGENT_OPENAI_API_KEY=nvapi-...
COLAB_AGENT_OPENAI_BASE_URL=https://integrate.api.nvidia.com/v1
```

Production Profile

```
COLAB_AGENT_ENV=prod
COLAB_AGENT_API_KEY=my-secret-key
COLAB_AGENT_DATABASE_URL=postgresql://user:pass@host/db
COLAB_AGENT_BUDGET_MAX_UNITS=500
```

Architecture



Module Map

Directory	Purpose
agent/	Core loop, LLM client, prompts, safety, memory, plugins
agent/plugin.py	Plugin base class, registry, hook runner (9

Directory	Purpose
	lifecycle hooks)
agent/llm_cache.py	SQLite-backed LLM response cache (thread-safe, WAL mode)
agent/self_improvement.py	Error pattern analysis + suggestion plugin
agent/circuit_breaker.py	Thread-safe circuit breaker with auto-recovery
agent/observability.py	Prometheus metrics, JSON logging
agent/health.py	Health reporter with auto-degraded status
agent/safety.py	Cost tracker, budget manager, code sanitization
agent/api.py	FastAPI app with /health, /metrics, /budget, /docs
agent/service.py	Stdlib HTTP server + FastAPI dual-mode entry point
capabilities/	13 plugin modules for Colab code generation
colab/	Executor, runtime management, secrets
config/	Pydantic v2 settings with env profiles (dev/staging/prod)
gh_integration/	GitHub API integration
models/	HuggingFace model management, fine-tuning orchestration
storage/	SQLAlchemy persistence (jobs, conversations, metrics)
ui/	Streamlit dashboard
tests/	500+ pytest tests across 20 test files

CLI Usage

```
colab-agent init          # Initialize DB + data dirs
colab-agent run TEXT      # Run agent with a goal
colab-agent interactive   # Interactive REPL mode
colab-agent serve         # Start FastAPI HTTP server (default)
colab-agent budget        # Show budget status
colab-agent list          # List recent jobs
colab-agent status ID     # Check job status
colab-agent logs ID       # View job logs
colab-agent models        # List registered models
colab-agent abort ID      # Abort a running job
colab-agent clean         # Clean up artifacts
```

Plugin System

Extend the agent with custom behaviour via 9 lifecycle hooks:

```
from agent.plugin import Plugin, plugin
```

```
@plugin(name="my_plugin", version="1.0.0", description="My custom behaviour")
class MyPlugin(Plugin):
    def before_plan(self, goal: str, context: dict) -> tuple[str, dict]:
        return f"[custom] {goal}", context

    def on_complete(self, result: dict, context: dict) -> tuple[dict, dict]:
        result["my_data"] = "logged"
        return result, context
```

Built-in plugins:

- **LLM Cache** (`agent/llm_cache.py`) — caches LLM responses to disk, auto-evicts by TTL
- **Self-Improvement** (`agent/self_improvement.py`) — analyzes failed runs, suggests prompt fixes

API Endpoints

When running `colab-agent serve` (FastAPI mode):

Endpoint	Description
GET /	Service info
GET /health	Agent health status, circuit state, uptime
GET /metrics	Prometheus-formatted metrics
GET /budget	Budget status, spend, remaining
GET /docs	Interactive OpenAPI documentation

HTTP API (stdlib fallback)

```
# Zero-dependency HTTP server
python -m agent.service

# FastAPI mode (requires uvicorn)
python -m agent.service --fastapi
```

Docker

```
# Build
docker build -t colab-agent .

# Run with PostgreSQL
docker-compose up
```

Backup & Restore

SQLite

```
# Backup
sqlite3 data/colab-agent.db ".backup 'backups/colab-agent-$(date +%F).db'"

# Restore
```

```
sqlite3 data/colab-agent.db ".restore 'backups/colab-agent-2026-01-01.db'"
```

PostgreSQL

```
# Backup
PGPASSWORD=$COLAB_AGENT_DATABASE_PASSWORD \
pg_dump -h $COLAB_AGENT_DATABASE_HOST -U $COLAB_AGENT_DATABASE_USER \
-d $COLAB_AGENT_DATABASE_NAME > backup_$(date +%F).sql

# Restore
PGPASSWORD=$COLAB_AGENT_DATABASE_PASSWORD \
psql -h $COLAB_AGENT_DATABASE_HOST -U $COLAB_AGENT_DATABASE_USER \
-d $COLAB_AGENT_DATABASE_NAME < backup_2026-01-01.sql
```

Data Retention

Data is auto-cleaned on service startup based on retention policies:

Category	Default TTL	Env override
Jobs	90 days	COLAB_AGENT_RETENTION_JOBS_DAYS
Conversations	90 days	COLAB_AGENT_RETENTION_CONVERSATIONS_DAYS
Training metrics	180 days	COLAB_AGENT_RETENTION_METRICS_DAYS
JSONL log files	30 days	COLAB_AGENT_RETENTION_LOGS_DAYS
Audit logs	365 days	COLAB_AGENT_RETENTION_AUDIT_LOG_DAYS
LLM cache	7 days	COLAB_AGENT_RETENTION_LLM_CACHE_DAYS

Monitoring

Sentry (optional)

Set `COLAB_AGENT_SENTRY_DSN` to enable error tracking. Optionally set `COLAB_AGENT_SENTRY_TRACES_RATE` (default `0.1`) for performance tracing.

Prometheus

The `/metrics` endpoint exposes counters and gauges in Prometheus text format. Configure your Prometheus server to scrape `http://<agent>:8000/metrics`.

Rate Limiting

FastAPI endpoints are rate-limited per-IP using an in-memory token bucket:

Variable	Default
COLAB_AGENT_RATE_LIMIT	60 requests/minute
COLAB_AGENT_RATE_LIMIT_BURST	10 burst

API Key Auth

When `COLAB_AGENT_API_KEY` is set, all endpoints except `/health` require `Authorization: Bearer <key>`. In production (`COLAB_AGENT_ENV=prod`), the API key is **required** and `/docs` is disabled.

Capabilities

The `capabilities/` module provides 13 plugin areas:

- **Data Collectors** — YouTube, Twitter, PDF, GitHub, Wikipedia, Slack, Arxiv, Reddit, CommonCrawl, Selenium
- **Data Processors** — Synthetic data gen, augmentation, privacy masking, toxicity filtering, dedup
- **Experiment Tracking** — MLflow/W&B integration, HP database, drift detection, ablation studies
- **Model Optimization** — LoRA adapter zoo, model merging, gradient checkpointing, mixed precision, FlashAttention, QAT
- **Training Utils** — Callbacks, LR schedulers, distributed training, checkpointing, evaluation, data collators
- **Runtime** — GPU detection, runtime switching, keep-alive, limits detection, benchmarking
- **GitHub** — Repo management, error commits, solution learning, PR creation, version tagging
- **Storage** — SQLite jobs, time-series metrics, model registry, artifact compression, HTML export, cross-session memory
- **Security** — Command blocklist, network logging, resource quotas, data leak detection, session encryption
- **UI** — Gradio chat, Streamlit dashboard, IPython widgets, terminal mode, approval modals
- **Autonomous Intelligence** — Paper reproduction, self-healing, feedback learning, meta-agent

Development

```
# Install dev deps
pip install -e ".[dev]"

# Lint
ruff check .
```

```
# Quick tests (no API calls, ~60s)
pytest tests/test_plugin.py tests/test_self_improvement.py tests/test_config.py
tests/test_api.py -v

# All unit tests (ignores API-dependent)
pytest tests/ --ignore=tests/test_models.py --ignore=tests/test_e2e_agent.py --
ignore=tests/test_integration.py --ignore=tests/test_storage.py -q

# Full suite
pytest tests/ -v --timeout=60
```

Production Hardening

All modules are hardened with:

- **Thread safety:** `threading.Lock` / `RLock` on all shared state
- **Input validation:** Pydantic models + defensive type checks
- **Error isolation:** Plugin exceptions don't crash the chain
- **Graceful degradation:** Cache survives DB errors, returns safe defaults
- **Resource cleanup:** `close()` methods, context managers, WAL mode
- **Structured logging:** JSONL format, contextual fields
- **Security:** Code sanitization, prompt injection detection, API key encryption (Fernet AES-128), audit log, circuit breaker
- **Safety limits:** Max 5 runtime switches, 10 retries/step, 50 total retries, 12h auto-shutdown, budget cap